EE 271 Digital Design
Final Project Proposal

Name: Martin Gong

Project: Flappy Bird

PROJECT DESCRIPTION

From a User Standpoint

This project implements the popular mobile game "Flappy Bird" on the DE1-SoC board using an 8×8 LED matrix display. The player controls a bird (represented by a red LED) that must navigate through a series of moving vertical barriers (represented by green LEDs). The bird is constantly affected by gravity, causing it to fall, and the player must press a button to make the bird "flap" upward.

User Interaction

- **KEY[0]:** When pressed, causes the bird to move upward (flap)
- **SW[9]:** Reset switch to restart the game
- **SW[1]:** Pause switch to temporarily freeze the game
- **SW[0]:** When activated, displays the high score on the HEX displays
- **HEX Displays (HEX0-HEX5):** Show current score during gameplay, or high score when SW[0] is on
- **8×8 LED Matrix:** Visual game display showing red LED for the bird's position and green LEDs for barrier columns with gaps to fly through

Game Mechanics

The bird continuously falls due to gravity simulation. When the player presses KEY[0], the bird moves up one position. Barriers scroll from right to left across the display, and the player must time their button presses to navigate through the gaps. The game ends if the bird collides with a barrier or flies out of bounds (top or bottom of the display). Score increments as barriers successfully pass the bird's position.

SYSTEM BLOCK DIAGRAM

The system is organized into several major subsystems:

1. Input Processing Block

- **Clock Divider:** Generates multiple clock frequencies from CLOCK_50
- **useInput:** Debounces KEY[0] input for reliable button detection
- **Timing Counters (counter1, counter2, counter3):** Generate enable signals for different game speeds

2. Bird Control Subsystem

- **Eight normalLight modules:** Control bird vertical position (one per LED row)
- **One centerLight module:** Handles middle position with initial spawn logic
- **bounds module:** Detects out-of-bounds conditions (top/bottom)
- **Outputs:** red_array[7:0][6] representing bird position in column 6

3. Barrier Generation Subsystem

- **startColumn:** Generates random barrier patterns with gaps for the bird to pass through
- **Eight normalColumn modules:** Shift barriers left across display (columns 0-7)
- **Outputs:** green_array[7:0][7:0] representing all barrier positions

4. Collision Detection

- **collision module:** Compares red_array and green_array to detect overlaps
- **Function:** Signals game over when bird position matches barrier position

5. Scoring Subsystem

- **score module:** Detects when barriers successfully pass bird position
- **Three increment modules:** Chain together to create 4-digit decimal counter (0-9999)
- **Display:** Outputs to HEX displays

6. Display and Pause Subsystem

- **pauseGame:** Overlays pause pattern on display when SW[1] is active
- **highscore:** Stores best score and displays it when SW[0] is on
- **matrix_driver:** Multiplexes red and green arrays to LED matrix via GPIO pins

IMPLEMENTATION DETAILS

Clock Management

All modules operate on a single clock domain (CLOCK_50) with divided clock signals generated by the clock_divider module. The system uses clk[14] as the main game clock for appropriate game speed. Enable signals from counter modules control update rates for different game elements:

- **input2:** Controls gravity frequency (bird falling rate)
- **enable2:** Controls barrier scrolling speed
- **enable3:** Controls new barrier generation rate

Data Structure

The design uses 2D array structures for both red (bird) and green (barriers) LEDs:

- **red_array[7:0][7:0]:** 8×8 array where only column [6] is used for bird position
- **green_array[7:0][7:0]:** Full 8×8 array representing barrier columns

This structure makes collision detection straightforward through bitwise comparison of red_array[row][6] with green_array[row][6].

Finite State Machine Design

Individual LED positions are controlled by FSM modules (normalLight, centerLight) that respond to:

- **Gravity signals:** Causing downward movement
- **User input:** Causing upward movement
- **Adjacent LED states:** To create continuous bird movement

Reset and Game Over

The system uses a compound reset signal: (reset | out_of_bounds | dead) that triggers when:

- **SW[9] is pressed:** Manual reset
- **Bird goes out of bounds:** Top or bottom
- **Bird collides with a barrier:** Collision detected

GPIO Pin Assignment

The LED matrix is driven through GPIO_0[35:0] with the following assignments:

- **GPIO_0[27:20]:** Red LED drivers (8 bits)
- **GPIO_0[35:28]:** Green LED drivers (8 bits)
- **GPIO_0[19:12]:** Row sink control for multiplexing (8 bits)

KEY FEATURES

- **Smooth Gravity Simulation:** Adjustable falling rate using counter-based timing
- **Responsive Controls:** Debounced input ensures reliable button response
- **Progressive Difficulty:** Continuous barrier generation keeps game challenging
- **Score Tracking:** Decimal score display up to 9999 with high score memory
- **Pause Functionality:** Game can be paused without losing state

- **Visual Feedback:** Clear LED matrix display distinguishes bird (red) from barriers (green)
- **Robust Design:** All modules follow single-clock methodology for reliable timing